BUSINESS CASE · APRILE 2026

Claude Registry

Come trasformare l'uso individuale di Claude Code in un asset condiviso, governato e misurabile per il team.

01 **Produttività scalabile.** Le capability migliori del team — oggi nelle teste dei senior — diventano disponibili a tutti, immediatamente.

02 **Qualità verificabile.** Ogni capability passa da PR, review e validazione automatica prima di arrivare ai progetti. Il comportamento di Claude è prevedibile e tracciabile.

03 **Conoscenza che non si perde.** Best practice, standard di codice e logiche di analisi sono nel registry — non nei file personali, non nelle teste delle persone.

STATO	CAPABILITY DISPONIBILI	COSTO INFRASTRUTTURA
Operativo — pronto per adozione	10 (3 stable, 7 beta)	€0 — usa tool già in uso

Il contesto: cosa stiamo facendo oggi

Il team usa Claude Code quotidianamente per sviluppo, analisi e revisione del codice. È già parte del flusso di lavoro. Il problema non è l'adozione dello strumento — **è che ogni persona lo usa in modo diverso, producendo risultati diversi.**

SENZA REGISTRY

- Ogni developer costruisce i propri prompt da zero
- Output di qualità variabile sullo stesso task
- Standard diversi per test, error handling, logging
- Il senior sa come istruire bene Claude, il junior no
- Cambio di convenzione → nessuno aggiorna i propri prompt
- Nuovo collega: ricomincia da zero

CON REGISTRY

- Capability pronte all'uso, install con un comando
- Output consistente per tutti i livelli di seniority
- Standard unificati: Java, Python, API, revisione
- Le istruzioni del senior sono lo standard di tutti
- Aggiornamento → PR, review, distribuzione automatica
- Nuovo collega: operativo in 30 minuti

⚠ **Il costo nascosto dell'assenza di struttura**

Non avere capability condivise non è una situazione "neutra" — è un costo attivo. Ogni developer passa tempo a costruire e ricostruire istruzioni per Claude, senza che questo sforzo produca valore condiviso. Moltiplicato per il team, per ogni sprint.

Cosa offre il Registry

Claude Registry è un repository Git con 10 **capability** già pronte — specializzazioni di Claude per ruoli tecnici specifici. Ogni capability incorpora gli standard del team: architettura pulita, pattern di test, gestione degli errori, sicurezza, osservabilità.

CAPABILITY	TIER	QUANDO USARLA
`software-architect`	STABLE	Analisi architetturale, ADR, valutazione trade-off, pattern di integrazione
`functional-analyst`	STABLE	Requisiti, use case, processi di business, acceptance criteria BDD
`developer-java-spring`	STABLE	Sviluppo Java/Spring Boot: architettura a strati, test, security, osservabilità
`technical-analyst`	BETA	Audit tecnico, debito tecnologico, vulnerabilità, CI/CD assessment
`developer-python`	BETA	Python/FastAPI: type hints, pydantic, pytest, structlog
`code-reviewer`	BETA	Code review strutturata: correttezza, sicurezza, test, manutenibilità
`test-writer`	BETA	Scrittura test JUnit 5, Mockito, Testcontainers, pytest
`debugger`	BETA	Diagnosi bug da stack trace e codice sorgente
`api-designer`	BETA	Design e review API REST, produzione spec OpenAPI 3.1
`documentation-writer`	BETA	README, runbook, guide API, overview architetturali

Le capability STABLE sono già in uso. Le BETA sono funzionanti ma in fase di affinamento sulla base del feedback del team.

I benefici in dettaglio

La seniority diventa scalabile

Oggi la differenza di output tra un developer senior e un junior usando Claude dipende principalmente da quanto sanno istruirlo. È conoscenza implicita, non trasferibile.

> *Con la capability `developer-java-spring`, un developer junior produce automaticamente codice con costructor injection, test JUnit 5 separati dal contesto Spring, exception handler RFC 7807, logging strutturato JSON — esattamente come un senior che segue le nostre linee guida. Non perché "sa" tutto — ma perché la capability lo guida.*

Onboarding accelerato

Un nuovo membro del team che deve iniziare a usare Claude Code secondo gli standard interni oggi richiede ore di allineamento. Con il registry, l'intera procedura è:

Clone registry → 1 comando setup → Claude Code aperto → Operativo ✓

Tutte le capability del team disponibili immediatamente, standard già incorporati.

Qualità consistente e governata

Ogni modifica a una capability passa attraverso una Pull Request su GitHub. Prima del merge, una pipeline automatica verifica struttura, completezza e correttezza del file. Poi è necessaria almeno una approvazione umana esplicita.

Il risultato: **tracciabilità completa** su chi ha modificato cosa, quando, con quale motivazione. Se una capability produce output indesiderato, si sa esattamente cosa correggere e come distribuire la correzione a tutto il team.

Conoscenza che non si perde

Le best practice di un collega esperto, oggi, esistono nei suoi file locali — o nella sua testa. Quando cambia progetto o lascia il team, spariscono.

Con il registry, ogni miglioramento che un developer apporta al proprio modo di usare Claude diventa una PR. Se approvata, entra nel patrimonio condiviso. **La conoscenza è nell'organizzazione, non nelle persone.**

Investimento e ritorno

Il registry è già operativo. Non ci sono costi di infrastruttura aggiuntivi: usa GitHub e Claude Code, già parte del nostro stack.

€0	30m	10
Costo infrastruttura aggiuntivo	Onboarding nuovo membro al registry	Capability disponibili oggi

Saving per il team

SCENARIO	SAVING STIMATO
Developer usa capability esistente invece di costruire prompt da zero	20–40 min per task
Eliminazione rework per output non conforme agli standard	1–2 ore per sprint a persona
Onboarding nuovo membro agli standard Claude del team	1–2 giorni lavorativi
Identificazione di regressioni nel comportamento di Claude	Variabile — evita escalation su progetti critici

ℹ **Stima conservativa per un team di 5 developer**

Con 2 capability usate per task al giorno e saving medio di 25 minuti a task, il risparmio è nell'ordine di **2–4 ore/settimana per persona**, pari a circa 1 giornata lavorativa a persona ogni 2 settimane. Senza considerare il valore della qualità standardizzata.

Come funziona — in una pagina

Per chi usa le capability

Un developer che vuole usare una capability esegue uno script:

```
$ ./setup-capabilities.sh /path/to/mio-progetto

  Seleziona le capability da installare:
  1 [stable] software-architect   ...
  2 [stable] developer-java-spring ...

  Selezione: 1 2

  ✓ software-architect installato
  ✓ developer-java-spring installato
  ✓ settings.json aggiornato
```

Da quel momento, Claude Code usa automaticamente la capability giusta in base al task.

Per chi contribuisce una nuova capability

1. **Crea lo scaffolding**
   Un comando crea branch, file, template e aggiorna il changelog automaticamente.
2. **Scrivi il system prompt**
   Definisci il ruolo, il formato di output, gli standard da applicare e le verifiche di qualità.
3. **Apri una Pull Request**
   GitHub Actions valida automaticamente e posta un commento con eventuali problemi. Nessun errore = pronta per review.
4. **Review e approvazione**
   Almeno un membro del team approva manualmente. Nessun merge senza consenso.
5. **Pubblicazione**
   Un comando pubblica la capability nel marketplace. Tutto il team può installarla.

Roadmap proposta

FASE	AZIONI	ORIZZONTE
Adozione immediata	Distribuzione delle 10 capability ai progetti attivi; raccolta feedback	Ora → Sprint +1
Stabilizzazione	Promozione capability beta → stable sulla base del feedback; fix prioritari	Sprint +2 → +4
Capability di dominio	Sviluppare capability specifiche per i nostri domini applicativi (es. compliance, finanza, migrazione legacy)	Trimestre +1
Allargamento del team	Coinvolgimento di altri team nell'uso e nella contribuzione al registry	Trimestre +2

In sintesi

Claude Code è già nel flusso di lavoro del team. Il valore che genera oggi dipende dal singolo developer — dalla sua esperienza, dal tempo che ha dedicato a costruire i propri prompt, dalla sua conoscenza degli standard interni.

Claude Registry risponde a una domanda semplice:

> *Vogliamo che questo valore resti frammentato per developer — o vogliamo che diventi un asset condiviso, governato e misurabile dell'organizzazione?*

L'infrastruttura è pronta. Le prime capability sono operative. Il passo successivo è l'adozione strutturata su tutti i progetti attivi.